

The XRPC API surface

All protocol XRPC methods for permissioned data are defined under one namespace: `com.atproto.space`. The proposal sorts them into four loose groups by what they concern:

- **Repo** methods concern an account's permissioned repo within a space, and are implemented by a repo host.
- **Host** methods concern a space as a whole, and are implemented by a space host.
- **PDS** methods are the required set of baseline PDS methods that applications can build against.
- **Syncer** methods concern a syncer getting real-time notifications of updates to a space or repos in a space.

Don't read the groups as a deployment diagram:

This grouping describes kinds of methods, not separate services. A single service (e.g. a PDS) is usually both a repo host for its accounts and a space host for the spaces anchored on it.

Permissioned Data proposal, "XRPC API"

In practice one PDS answers repo methods for the repos it stores, host methods for the spaces anchored on its accounts' DIDs, and the PDS methods for its own users. Nothing requires that arrangement, though. A permissioned repo or a space may be hosted by any service that implements the required APIs.

The full method table

The table below reproduces the proposal's method list. The Auth column is where most of the design lives, so read it closely.

Method	Role	Type	Auth	Description
<code>getSpace</code>	host	query	space credential	Describe a space, including an open-union <code>config</code> carrying host-specific configuration.
<code>getSpaceCredential</code>	host	procedure	delegation token (+ client attestation)	Exchange a delegation token for a space credential. A client attestation is also required when the space g ? on client app identity.

Method	Role	Type	Auth	Description
<code>listRepos</code>	host	query	space credential	List the known repos that hold data in a space, with each repo's current <code>rev</code> and commit <code>hash</code> .
<code>getRecord</code>	repo	query	OAuth / space credential	Fetch a single record's value.
<code>listRecords</code>	repo	query	OAuth / space credential	List the records in a repo, inlining record values by default. Set <code>excludeValues</code> for a metadata-only listing.
<code>getBlob</code>	repo	query	OAuth / space credential	Fetch a blob by CID.
<code>getLatestCommit</code>	repo	query	OAuth / space credential	The current signed commit for a repo.
<code>getRepo</code>	repo	query	OAuth / space credential	Download a whole repo as a serialized CAR for full-state backfill.
<code>listRepoOps</code>	repo	query	OAuth / space credential	Primary sync mechanism. A repo's operation log since a given revision, inlining record values by default. Set <code>excludeValues</code> for metadata-only entries.
<code>getDelegationToken</code>	pds	query	OAuth	Mint a delegation token for a space. Served by the requesting user's PDS.
<code>createRecord</code>	pds	procedure	OAuth	Create a record in the caller's permissioned repo for a space.
<code>putRecord</code>	pds	procedure	OAuth	Create or update a record.
<code>deleteRecord</code>	pds	procedure	OAuth	Delete a record.
<code>applyWrites</code>	pds	procedure	OAuth	Apply a batch of creates, updates, and deletes to one repo atomically.
<code>listSpaces</code>	pds	query	OAuth	The spaces the caller holds a repo in.
<code>registerNotify</code>	repo/host	procedure	space credential	Register an endpoint to be notified of writes. On the space host, subscribes to the whole space. On a repo host with a <code>repo</code> , subscribes to that repo.
<code>notifyWrite</code>	syncer/host	procedure	service auth	Notify that a repo advanced. Sent by a repo host to the space host, and forwarded by the space host to registered syncers.

Method	Role	Type	Auth	Description
<code>notifySpaceDeleted</code>	syncer/repo	procedure	service auth	Notify that a space was deleted. Sent by the authority to members and registered syncers.

Reading the Auth column

The API uses four kinds of authentication, and each method takes the kind that matches who is calling and why.

The write methods (`createRecord`, `putRecord`, `deleteRecord`, `applyWrites`), plus `getDelegationToken` and `listSpaces`, take OAuth only. These are calls an application makes to its own user's PDS, on that user's behalf. Writes in particular can never ride on a space credential:

A **space credential** grants whole-space read/sync access directly, so the read and sync methods accept **either** a covering OAuth scope or a space credential. Write methods accept only an OAuth credential, since a write is attributed to the authoring user.

Permissioned Data proposal, "Read access" (under "OAuth scopes")

The read and sync methods on a repo host (`getRecord`, `listRecords`, `getBlob`, `getLatestCommit`, `getRepo`, `listRepoOps`) take either. A user's own app can read the user's own repo with just a covering OAuth scope. Anyone else reading a repo in the space presents a space credential, which grants whole-space read/sync access directly.

The space-wide host queries `getSpace` and `listRepos`, and the subscription procedure `registerNotify`, take a space credential. You have to already be admitted to the space before you can describe it, enumerate its writer set, or subscribe to its write notifications.

The notification procedures `notifyWrite` and `notifySpaceDeleted` take service auth. No end user is party to these calls. A repo host tells the space host a repo advanced, the space host fans that out to registered syncers, and an authority announces a space's deletion. Services authenticate to each other with service auth tokens.

That leaves `getSpaceCredential`, which fits none of those patterns because it sits at the boundary where credentials get created. Its input is a **delegation token** minted by the user's PDS, proving the app acts for that user, plus a **client attestation** if the space gates on client app identity. Its output is the space credential itself. The application can't authenticate this call with a space credential, since that is the thing it's asking for. Its OAuth token is no use here either: that token means something to the user's own PDS, not to a foreign space authority.

simplespace: the required PDS space management

The core protocol deliberately doesn't say how spaces get created or how an authority decides who may read them. Those decisions belong to a **space-management implementation**, a layer that sits above the protocol and is identified by its own lexicon namespace. The proposal defines one such implementation that is guaranteed to exist everywhere:

`com.atproto.simplespace` is the space-management implementation that every PDS MUST support. It gives applications a baseline that is available on every account's PDS to build against. `simplespace` spaces are anchored on a user's own DID and governed by an explicit member list (or the `public` and `managing-app` policies described below).

Permissioned Data proposal, "Required PDS space management: `simplespace`"

Required does not mean exclusive, and it does not mean special:

`simplespace` is neither the only permitted implementation nor a privileged one. It is simply the one that PDSs are required to support. Other space types may define their own management implementations and are full protocol participants, but they are hosted on bespoke space services rather than on the PDS.

Permissioned Data proposal, "Required PDS space management: `simplespace`"

Its administrative surface is driven by OAuth, not by space credentials: "All `simplespace` methods are called with an OAuth credential with the relevant `manage` scope." This is what the `manage` verbs from the `space:` scope grammar concretely buy you. In `com.atproto.simplespace`, `manage=update` authorizes `com.atproto.simplespace.updateSpace` as well as `addMember` and `removeMember`.

The `simplespace` methods

Method	Type	Description
<code>createSpace</code>	procedure	Create a space (caller becomes the authority)
<code>updateSpace</code>	procedure	Update config
<code>deleteSpace</code>	procedure	Delete the space
<code>addMember</code>	procedure	Add a member (by DID) to view the space.
<code>removeMember</code>	procedure	Remove a member (by DID).
<code>listMembers</code>	query	List the current members of a space.

Configuration: `policy`, `appAccess`, `managingApp`

A `simplespace` space is configured along three dimensions. Configuration is updated through `updateSpace` and surfaced, to anyone with access to the space, through `com.atproto.space.getSpace`.

Field	Values	Description
<code>policy</code>	<code>public</code> <code>member-list</code> <code>managing-app</code>	How the authority decides whether to authorize a <i>user</i> .
<code>appAccess</code>	open union (<code>#open</code> <code>#allowList</code>)	How the authority decides whether to authorize an <i>app</i> .
<code>managingApp</code>	service identifier (DID + fragment)	Used to route application requests, and as the access check target in <code>managing-app</code> mode.

The sentence that ties the two axes together is a conjunction:

A user must be authorized by the `policy` **and** their app by `appAccess` for a credential to be minted. A syncing app needs a valid delegation token regardless.

Permissioned Data proposal, "Configuration" (under "Required PDS space management: `simplespace`")

The axes are evaluated independently, and a credential is minted only when both pass. A user on the member list gets nothing if her client fails the app check. An allow-listed app gets nothing for a user the policy rejects. And the tokens settle identity, not authorization: a valid delegation token and a valid client attestation establish who the user and the app are, while these two config fields govern whether that pair gets in.

`policy` decides per-user authorization:

- `member-list` (the default): authorize requesters present on the member list.
- `public`: authorize any requester.
- `managing-app`: at mint time, ask the `managingApp` whether to authorize the request, via `checkUserAccess` (below). This enables dynamic policies (e.g. follower-gating) without the app maintaining a list.

`appAccess` decides per-app authorization. It's an open union with two current variants:

- `#open` (the default): any application may access the space. No client attestation is required, so public clients work.
- `#allowList`: only the apps named in `allowed` may access the space.

What makes `#allowList` more than a suggestion is *which* identifier it gets checked against:

`#allowList`: only the apps named in `allowed` may access the space. The authority evaluates the list against the **attested** `client_id` (the `iss` of the verified client attestation), so it is enforceable rather than advisory.

Permissioned Data proposal, "Configuration" (under "Required PDS space management: `simplespace`")

The attested `client_id` is the `iss` of a client attestation the authority has verified itself: it resolves that `client_id` to the client's published `client-metadata.json`, fetches the JWKS, and checks the signature. A self-reported identifier could be claimed by anyone; a verified one can't. The delegation token doesn't help here, and it's a different token entirely. Its `iss` is the *user's* DID, and it says nothing about which application is acting.

The managing app and `checkUserAccess`

Spaces may list a `managingApp`, a service identifier (DID + fragment). Any account with access to the space can read it out of the config via `getSpace`, and it can be used to route application-level requests a generic PDS cannot handle. Join requests are the obvious example. Calls to managing apps are generally authenticated using service auth tokens.

When the space's `policy` is `managing-app`, the authority defers the per-user decision to that app at credential-mint time by calling `com.atproto.simplespace.checkUserAccess`. This method is the odd one out in the namespace:

Unlike the other `simplespace` methods, `checkUserAccess` is served by the `managingApp`, not the PDS. The authority calls it with itself as `iss` and the `managingApp`'s service identifier as `aud`, so the app can verify the call genuinely originates from the space's authority.

Permissioned Data proposal, "The managing app"

The direction of the call is easy to get backwards. The *authority* calls the *managing app*, not the other way around, and the client application is not involved. The authority passes the space, the requesting user, and the requesting client (the attested `client_id`, if any). The app evaluates the request against whatever application-layer state it maintains, like follower graphs, paid-subscription status, or join approvals, and returns its decision. The authority mints the credential only if the app authorizes, and it still applies `appAccess` as usual: `managing-app` replaces the `policy` axis only.

Moderation

Moderation keeps the public-broadcast model: labeling plus infrastructure takedowns, with every service moderating what it hosts and serves per its legal requirements, terms of service, and community guidelines. The one obvious difference is that the data is not open, so a moderation service cannot observe a space it has not been admitted to. A moderation service functions as just another reader. To label content in a space, it must hold a space credential like any other syncer, which means the space authority has admitted it under the same access-control rules as everyone else.

Where the labels then go is the part that changes:

A moderation service should not publish public labels for records in permissioned spaces, as this leaks metadata about otherwise-private data. The public

`com.atproto.label.subscribeLabels` endpoint is therefore a poor fit here. Instead, labelers may publish labels as records in a permissioned repo within the relevant space, keeping the labels inside the same access boundary as the content they describe.

Permissioned Data proposal, "Moderation" (under "Considerations")

Even a label that names an `at://` URI without quoting content reveals that the record exists and drew a labeler's attention. So labels stay inside the boundary.

At the infrastructure level the familiar levers apply. Each participant retains authority over their own repo and can delete their own records. A repo host can take down accounts or refuse to serve repos it hosts. An application can filter records from the views it serves. A space authority holds one additional lever with no public analogue: since reading requires a credential it issues, it can decline to issue one to a user and cut off that user's read access, or stop issuing credentials for the space entirely. Some spaces may also include records with application-level semantics for restricting writes from moderated users.

Scaling

The scaling story is expected to resemble public broadcast: data is partitioned per user across many hosts, no single node at the hosting layer holds an entire space, and applications choose how much and which data to index. The one major difference is the **lack of relays**. Permissioned repos are by nature non-rebroadcastable, so there is no collated firehose. Applications sync repos directly from PDSes, which places additional load on PDSes. The proposal gives three reasons this is manageable.

First, the sync protocol itself is significantly lighter-weight than public sync, since it does not carry the overhead of maintaining and transmitting the structural nodes of the MST.

Second, load scales with applications, not end users:

Second, sync load scales with the number of applications syncing a space, not the number of end users. An application serving many users pulls each repo once and fans it out to them from its own copy, which keeps the number of distinct syncers per repo relatively low.

Permissioned Data proposal, "Scaling" (under "Considerations")

Third, the sync APIs are not open to the public. Each sync method requires a space credential, which is generally issued only to a closed set of syncers, and can be correlated across requests to enforce rate limits and other controls. Resource abuse is much easier to mitigate than on an open endpoint.

Space deletion

A space may be deleted by its authority. The authority stops issuing credentials, no longer answers for the space, and deletes its own repo in the space. It then notifies registered syncers and the known repo hosts of participating members with `com.atproto.space.notifySpaceDeleted`, over the same best-effort path as write notifications.

What each recipient must do splits by role, and the split follows data ownership:

A **syncer** should delete every copy of the space's data it holds, both the repos it pulled and any derived state, as it is no longer authorized to retain them. A **repo host** should instead flag the member's repo as belonging to a deleted space rather than erase it. The data is the user's own, so the repo host decides how and whether to surface it to the user (e.g. an export, a grace period) before garbage-collecting.

Permissioned Data proposal, "Space deletion"

A syncer holds *other people's* data purely by grace of a credential that will never be renewed, so its copies must go, indexes and any other derived state included. A repo host holds *its own user's* records. Deleting the space entitles nobody to destroy the member's writing, so the host flags the repo and gives the user a path out (an export, a grace period) before garbage collection.

Account lifecycle

An account's participation in permissioned data is tied to the same DID and signing key as its public atproto identity, so the existing lifecycle events (migration, key rotation, deactivation, and deletion) all function the same way here.

Migration

Moving a permissioned repo between hosts works the same as migrating a public repository. The main difference is multiplicity: a user has *many* permissioned repos rather than one, so account migration flows must enumerate and track all of an account's permissioned repos and the blobs associated with them. Recall from the sync section that migration also resets the oplog. The new host starts its operation log afresh, and syncers that cannot find their `since` revision fall back to full-state recovery.

Deactivation and deletion

These function exactly as they do for public broadcast. If an account is **deleted**, downstream services are expected to delete all public and permissioned data associated with the account. If an account is **deactivated**, downstream services are expected to stop serving all public and permissioned data associated with it.

Identity and account events

Applications learn about changes to an account's status or identity, such as its signing key and handle, via events on the firehose (`com.atproto.sync.subscribeRepos`), and those updates apply to permissioned repos in exactly the same manner as public ones. That produces a counterintuitive requirement for permissioned-only applications:

This means an application that syncs only permissioned repos, and no public repositories, still needs to subscribe to a firehose to receive `#account` and `#identity` events. Future work may include an additional subscription endpoint that broadcasts only those two event types without the full stream of public repository commits.

Permissioned Data proposal, "Account lifecycle" (under "Considerations")

The firehose carries no permissioned repo commits. There is no relay for permissioned data, and write notifications travel via `registerNotify` and `notifyWrite` instead. The subscription exists solely so the application sees key rotations, handle changes, and account status changes for the DIDs whose permissioned repos it syncs.

You should now be able to explain

- The four method groups of the `com.atproto.space` API (repo, host, PDS, syncer) and why they describe kinds of methods rather than separate services
- Which authentication each class of method takes: OAuth for writes and PDS methods, OAuth or a space credential for read/sync methods, a space credential for the space-wide host queries and `registerNotify`, and service auth for `notifyWrite` and `notifySpaceDeleted`
- Why write methods accept only an OAuth credential, never a space credential
- What `getSpaceCredential` takes as input (a delegation token, plus a client attestation when the space gates on client app identity) and what it returns
- What `simplespace` is: the space-management implementation every PDS must support, anchored on the user's own DID, driven by OAuth `manage` scopes, and neither the only permitted implementation nor a privileged one
- The six `simplespace` methods and the three configuration fields (`policy`, `appAccess`, `managingApp`)
- Why a credential is minted only when the user passes `policy` AND the app passes `appAccess`
- What `#allowList` is evaluated against (the attested `client_id`, the `iss` of the verified client attestation) and why that makes it enforceable rather than advisory
- How `checkUserAccess` works: served by the `managingApp`, called by the space authority at mint time with itself as `iss` and the app's service identifier as `aud`
- How moderation works in permissioned spaces: labelers are ordinary credentialed readers, and labels are published as records inside the space rather than as public labels, which would leak metadata
- The moderation levers at each level, including the authority's unique lever of declining to issue credentials

- Why permissioned sync scales: no relay, but lighter-than-MST sync, load proportional to syncing applications rather than end users, and credential-gated APIs that permit rate limiting
- What happens on space deletion, and why a syncer must delete everything (including derived state) while a repo host flags the member's repo instead of erasing it
- How account migration, deactivation, deletion, and identity events carry over to permissioned data, including why a permissioned-only application still needs a firehose subscription for `#account` and `#identity` events